

Software metric

In software engineering and development, a **software metric** is a standard of measure of a degree to which a software system or process possesses some property.^{[1][2]} Even if a metric is not a measurement (metrics are functions, while measurements are the numbers obtained by the application of metrics), often the two terms are used as synonyms. Since quantitative measurements are essential in all sciences, there is a continuous effort by computer science practitioners and theoreticians to bring similar approaches to software development. The goal is obtaining objective, reproducible and quantifiable measurements, which may have numerous valuable applications in schedule and budget planning, cost estimation, quality assurance, testing, software debugging, software performance optimization, and optimal personnel task assignments.

Common software measurements

Common software measurements include:

- ABC Software Metric
- Balanced scorecard
- Bugs per line of code
- Code coverage
- Cohesion
- Comment density^[3]
- Connascent software components
- Constructive Cost Model
- Coupling
- Cyclomatic complexity (McCabe's complexity)
- Cyclomatic complexity density^{[4][5]}
- Defect density - defects found in a component
- Defect potential - expected number of defects in a particular component
- Defect removal rate
- DSQI (design structure quality index)
- Function Points and Automated Function Points, an Object Management Group standard^[6]
- Halstead Complexity
- Instruction path length
- Maintainability index
- Source lines of code - number of lines of code
- Program execution time
- Program load time
- Program size (binary)
- WEAVE score
- Weighted Micro Function Points
- Cycle time (software)

- First pass yield
- Corrective Commit Probability^[7]

Limitations

As software development is a complex process, with high variance on both methodologies and objectives, it is difficult to define or measure software qualities and quantities and to determine a valid and concurrent measurement metric, especially when making such a prediction prior to the detail design. Another source of difficulty and debate is in determining which metrics matter, and what they mean.^{[8][9]} Researchers argue that commonly used software metrics, promoted frequently in academic research, are not practically useful for software professionals' decision-making.^[10] The practical utility of software measurements has therefore been limited to the following domains:

- Scheduling
- Software sizing
- Programming complexity
- Software development effort estimation
- Software quality

A specific measurement may target one or more of the above aspects, or the balance between them, for example as an indicator of team motivation or project performance.

Additionally metrics vary between static and dynamic program code, as well as for object oriented software (systems).^{[11][12]}

Acceptance and public opinion

Some software development practitioners point out that simplistic measurements can cause more harm than good.^[13] Others have noted that metrics have become an integral part of the software development process.^[8] Impact of measurement on programmer psychology have raised concerns for harmful effects to performance due to stress, performance anxiety, and attempts to cheat the metrics, while others find it to have positive impact on developers value towards their own work, and prevent them being undervalued. Some argue that the definition of many measurement methodologies are imprecise, and consequently it is often unclear how tools for computing them arrive at a particular result,^[14] while others argue that imperfect quantification is better than none (“You can’t control what you can't measure.”).^[15] Evidence shows that software metrics are being widely used by government agencies, the US military, NASA,^[16] IT consultants, academic institutions,^[17] and commercial and academic development estimation software.

Further reading

- J. Smith, *Introduction to Linear Programming*, Acme Press, 2010. An introductory text.
- Reijo M.Savola, *Quality of security metrics and measurements*, *Computers & Security*, Volume 37, September 2013, Pages 78-90.^[18]

See also

- [Goal Question-Metric](#)
- [List of tools for static code analysis](#)
- [Orthogonal Defect Classification](#)
- [Software engineering](#)
- [Software package metrics](#)

References

1. Fenton, Norman E. (2014). *Software metrics : a rigorous and practical approach*. James Bieman (3rd ed.). Boca Raton, FL. ISBN 978-1-4398-3823-5. OCLC 834978252 (<https://search.worldcat.org/oclc/834978252>).
2. Timóteo, Aline Lopes; Álvaro, Re; Almeida, Eduardo Santana De; De, Silvio Romero; Meira, Lemos. *Software Metrics: A Survey*. CiteSeerX 10.1.1.544.2164 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.544.2164>).
3. "Descriptive Information (DI) Metric Thresholds" (http://www.lsec.dnd.ca/qsd_current_version/eng_support/di/metrics.htm). *Land Software Engineering Centre*. Retrieved 19 October 2010.
4. Gill, G. K.; Kemerer, C. F. (December 1991). "Cyclomatic complexity density and software maintenance productivity". *IEEE Transactions on Software Engineering*. **17** (12): 1284–1288. doi:10.1109/32.106988 (<https://doi.org/10.1109%2F32.106988>). ISSN 1939-3520 (<http://search.worldcat.org/issn/1939-3520>).
5. "maintainability - Does it make sense to compute cyclomatic complexity/lines of code ratio?" (<https://softwareengineering.stackexchange.com/questions/56056/does-it-make-sense-to-compute-cyclomatic-complexity-lines-of-code-ratio>). *Software Engineering Stack Exchange*. Retrieved 2021-03-01.
6. "OMG Adopts Automated Function Point Specification" (http://www.omg.org/news/releases/p_r2013/01-17-13.htm). Omg.org. 2013-01-17. Retrieved 2013-05-19.
7. Amit, Idan; Feitelson, Dror G. (2020-07-21). "The Corrective Commit Probability Code Quality Metric". arXiv:2007.10912 (<https://arxiv.org/abs/2007.10912>) [cs.SE (<https://arxiv.org/archive/cs/SE>)].
8. Binstock, Andrew (March 2010). "Integration Watch: Using metrics effectively" (<http://www.sdtimes.com/link/34157>). *SD Times*. BZ Media. Retrieved 19 October 2010.
9. Kolawa, Adam (7 August 2008). "When, Why, and How: Code Analysis" (<https://www.codeproject.com/Articles/28440/When-Why-and-How-Code-Analysis>). *The Code Project*. Retrieved 14 February 2021.
10. Tempero, Ewan; Ralph, Paul (2026-03-16). "Making Software Metrics Useful" (<https://arxiv.org/abs/2603.16012v1>). *arXiv.org*. Retrieved 2026-03-20.
11. Gosain, Anjana; Sharma, Ganga (2015). "Dynamic Software Metrics for Object Oriented Software: A Review" (https://link.springer.com/chapter/10.1007/978-81-322-2247-7_59). In Mandal, J. K.; Satapathy, Suresh Chandra; Kumar Sanyal, Manas; Sarkar, Partha Pratim; Mukhopadhyay, Anirban (eds.). *Information Systems Design and Intelligent Applications*. Advances in Intelligent Systems and Computing. Vol. 340. New Delhi: Springer India. pp. 579–589. doi:10.1007/978-81-322-2247-7_59 (https://doi.org/10.1007%2F978-81-322-2247-7_59). ISBN 978-81-322-2247-7.
12. S, Parvinder Singh; Singh, Gurdev. *Dynamic Metrics for Polymorphism in Object Oriented Systems*. CiteSeerX 10.1.1.193.4307 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.193.4307>).

13. Kaner, Dr. Cem (2004), *Software Engineer Metrics: What do they measure and how do we know?*, CiteSeerX 10.1.1.1.2542 (<https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.1.2542>)
14. Lincke, Rüdiger; Lundberg, Jonas; Löwe, Welf (2008), "Comparing software metrics tools" (<http://www.arisa.se/files/LLL-08.pdf>) (PDF), *International Symposium on Software Testing and Analysis 2008*, pp. 131–142
15. DeMarco, Tom (1982). *Controlling Software Projects: Management, Measurement and Estimation*. Yourdon Press. ISBN 0-13-171711-1.
16. "NASA Metrics Planning and Reporting Working Group (MPARWG)" (<https://web.archive.org/web/20111022023020/https://earthdata.nasa.gov/our-community/esdswg/metrics-planning-and-reporting-mparwg>). Earthdata.nasa.gov. Archived from the original (<https://earthdata.nasa.gov/our-community/esdswg/metrics-planning-and-reporting-mparwg>) on 2011-10-22. Retrieved 2013-05-19.
17. "USC Center for Systems and Software Engineering" (http://sunset.usc.edu/csse/research/COCOMOII/cocomo_main.html). Sunset.usc.edu. Retrieved 2013-05-19.
18. Savola, Reijo M. (2013-09-01). "Quality of security metrics and measurements" (<https://www.sciencedirect.com/science/article/pii/S0167404813000850>). *Computers & Security*. **37**: 78–90. doi:10.1016/j.cose.2013.05.002 (<https://doi.org/10.1016%2Fj.cose.2013.05.002>). ISSN 0167-4048 (<https://search.worldcat.org/issn/0167-4048>).

External links

- [Software Metrics \(http://www.sqa.net/softwarequalitymetrics.html\)](http://www.sqa.net/softwarequalitymetrics.html) (SQA.net)
- [Software Engineering Metrics: What do they measure and how do we know \(http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.1.2542\)](http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.1.2542)
- [NASA Standard NASA-STD-8739.8 \(Software Assurance and Software Safety Standard\) \(https://web.archive.org/web/20201016160155/https://standards.nasa.gov/standard/osma/nasa-std-87398\)](https://web.archive.org/web/20201016160155/https://standards.nasa.gov/standard/osma/nasa-std-87398)
- [HIS Source Code Metrics \(https://emenda.com/his/\)](https://emenda.com/his/) (*outdated but for reference*; related see AUTOSAR)
- [HIS Source Code Metrics version 1.3.1 01.04.2008 \(https://www.exida.com/Blog/software-metrics-iso-26262-iec-61508\)](https://www.exida.com/Blog/software-metrics-iso-26262-iec-61508) (*outdated but for reference*; related see AUTOSAR)
- [A framework for source code metrics \(https://dl.acm.org/doi/10.1145/1839379.1839400\)](https://dl.acm.org/doi/10.1145/1839379.1839400)
- [NASA.gov \(https://swehb.nasa.gov/display/SWEHBVB/Metrics+-+Software+Metrics+Report\)](https://swehb.nasa.gov/display/SWEHBVB/Metrics+-+Software+Metrics+Report)
- [SonarQube Metric Definitions \(https://docs.sonarqube.org/latest/user-guide/metric-definitions/\)](https://docs.sonarqube.org/latest/user-guide/metric-definitions/)
- [Metrics of Object Oriented Software \(https://www.researchgate.net/publication/235759663_Metrics_of_Object_Oriented_Software/stats\)](https://www.researchgate.net/publication/235759663_Metrics_of_Object_Oriented_Software/stats) (2010)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Software_metric&oldid=1364790696"